

DESAFIO TÉCNICO

Desenvolvedor(a) Sênior Full Stack

Tribe Lyceum - Techne

Prazo de entrega	7 dias corridos
Dedicação esperada	8 a 16 horas reais de dedicação
Stack obrigatória	Backend em Java com Spring Boot. Frontend em Angular, JavaScript, TypeScript ou framework web equivalente.
Domínio do desafio	Sistema acadêmico de matrículas, alinhado a modularização, desacoplamento, IA, observabilidade e evolução de legado.
Entrega final	Link do repositório com README completo, instruções de execução, testes e principais decisões técnicas.

Bem-vindo(a) ao desafio técnico

Este desafio faz parte do processo seletivo para Desenvolvimento Sênior. Mais do que validar apenas código funcional, queremos entender sua abordagem arquitetural, sua capacidade de desacoplamento, sua atenção a consistência, observabilidade, testes, documentação e tomada de decisão técnica.

01 Instruções gerais

Leia esta seção antes de iniciar a implementação.

1. Você deve desenvolver a solução e disponibilizar o código em um repositório acessível para avaliação.
2. O prazo de entrega é de 7 dias corridos a partir do envio do desafio.
3. Ao finalizar, envie o link do repositório com README contendo instruções de execução, testes, tecnologias utilizadas, principais decisões técnicas e uso de IA.
4. A solução precisa ser executável de forma reproduzível. Para o nível Sênior, o Docker Compose deve subir aplicação, banco de dados e mensageria.
5. A entrevista técnica, quando houver continuidade no processo, será baseada no código entregue. Você deverá explicar decisões, regras críticas, testes e eventual uso de IA.

► Uso de IA

O uso de ferramentas de IA é permitido e incentivado. Informe no README quais ferramentas utilizou, em quais partes do projeto, quais decisões revisou manualmente e quais trechos considera mais críticos.

O uso de IA não será avaliado negativamente. O ponto crítico é a sua capacidade de explicar o código, as decisões tomadas e os trade-offs assumidos.

02 Desafio funcional

Base comum do desafio técnico.

Desenvolva uma solução para gestão de matrículas acadêmicas. A solução deve permitir cadastrar alunos, cursos, disciplinas e turmas, além de controlar matrículas com regras de negócio claras.

► Entidades sugeridas

- Aluno
- Curso
- Disciplina
- Turma
- Matrícula

► Regras de negócio mínimas

- Um aluno só pode ser matriculado em turmas abertas.
- Uma turma possui limite de vagas.
- Um aluno não pode se matricular duas vezes na mesma turma.
- Uma matrícula possui status: PENDENTE, CONFIRMADA ou CANCELADA.
- Ao confirmar uma matrícula, a vaga da turma deve ser consumida.
- Ao cancelar uma matrícula confirmada, a vaga deve ser liberada.
- Deve haver consulta de matrículas por aluno e por turma.

03 Base técnica obrigatória

Itens esperados como fundação para a entrega Sênior.

Como a entrega é para nível Sênior, ela deve contemplar os requisitos funcionais e técnicos de base esperados para uma aplicação sustentável.

Área	Expectativa
Backend	Java com Spring Boot; API REST funcional; separação clara entre controller, service/application, domain/model, repository/persistence e DTOs.
Persistência	Banco de dados relacional com JPA/Hibernate e migrations com Flyway ou Liquibase.
Funcionalidades	Cadastro, edição, listagem e exclusão de alunos, cursos, disciplinas e turmas; matrícula de aluno em turma; consultas de matrículas por aluno e por turma.
Validações e erros	Validação de entrada nas APIs e tratamento padronizado de erros.
Testes	Testes automatizados cobrindo as principais regras de matrícula, incluindo testes unitários e testes de integração ou testes de API.
Frontend	Frontend estruturado com componentes, telas separadas, tratamento de erros e consumo organizado da API.
Documentação	Documentação da API com Swagger/OpenAPI e README com execução, testes, decisões técnicas, tecnologias usadas e uso de IA.

Área	Expectativa
Ambiente	Docker Compose com banco de dados. Para Sênior, o Compose também deve contemplar mensageria e aplicação.

04 Requisitos específicos para Dev Sênior

Arquitetura, desacoplamento, mensageria, consistência e evolução técnica.

Objetivo: avaliar arquitetura, desacoplamento, mensageria, consistência, observabilidade, testes, tomada de decisão e capacidade de evolução técnica.

- Solução modular ou baseada em mais de um serviço.
- Separação clara entre contexto acadêmico e contexto de notificações, auditoria ou relatórios.
- Comunicação assíncrona usando RabbitMQ, Kafka ou tecnologia equivalente.
- Publicação de eventos de domínio, como MatriculaCriada, MatriculaConfirmada e MatriculaCancelada.
- Consumidor de eventos em outro módulo ou serviço.
- Garantia de consistência na regra de vagas.
- Preocupação explícita com concorrência na matrícula.
- Testes unitários e de integração nas regras críticas.
- Docker Compose subindo aplicação, banco de dados e mensageria.
- Observabilidade mínima: logs estruturados, correlation ID ou trace ID, health checks e métricas básicas quando possível.
- Documentação arquitetural curta explicando decisões, trade-offs, riscos conhecidos, evolução da solução e tratamento de falhas na mensageria.
- README completo.

► Resumo do que esperamos encontrar no repositório

Entrega obrigatória	O que vai te destacar
• README com instruções de execução. • Testes unitários e de integração. • Docker Compose completo. • Eventos de domínio publicados e consumidos. • Controle de concorrência nas vagas. • Logs estruturados e correlation ID ou trace ID. • Documentação arquitetural com decisões e trade-offs.	• Arquitetura coesa e justificada. • Separação real de contextos. • Tratamento de falhas na mensageria. • Consistência entre README, arquitetura e implementação. • Clareza na evolução prevista da solução. • Uso de IA documentado e revisado criticamente. • Código que você consiga explicar com segurança.

05 README esperado

O README é parte essencial da avaliação.

- Como rodar o projeto localmente.
- Como subir a solução com Docker Compose.
- Como executar os testes automatizados.
- Quais tecnologias foram usadas.
- Quais foram as principais decisões técnicas.
- Como a regra de vagas foi protegida.
- Como a solução trata concorrência na matrícula.
- Como os eventos de domínio são publicados e consumidos.

- Como a solução trata falhas na mensageria.
- Quais logs, health checks, métricas ou mecanismos de rastreabilidade foram implementados.
- Quais ferramentas de IA foram utilizadas, em quais partes, o que foi revisado manualmente e quais trechos são mais críticos.

06 Critérios críticos de avaliação

Itens que podem impedir a continuidade no processo.

Tema	Ponto crítico
Execução	A aplicação não pode ser executada de forma reproduzível; não há instrução clara de execução; o projeto não roda.
Stack	O backend não foi desenvolvido em Spring Boot.
Persistência	Não há persistência de dados.
Regras de negócio	A regra de matrícula não está implementada ou a regra de vagas pode ser quebrada facilmente.
Camadas	Não há separação clara de responsabilidades; o código está concentrado em controllers.
Testes	Não há testes para regras críticas de matrícula.
Erros	Não há tratamento adequado e padronizado de erros.
Mensageria	Não há mensageria real ou não há consumidor de eventos em outro módulo ou serviço.
Arquitetura	A separação em serviços ou módulos é artificial e sem ganho arquitetural.
Consistência	Não há preocupação com consistência ou concorrência.
Documentação	Não há documentação arquitetural ou README suficiente para executar e validar a solução.
Entrevista	O candidato não consegue explicar as decisões técnicas, o código entregue ou o uso de IA.

07 Diferenciais

Itens não obrigatórios, mas valorizados para o nível Sênior.

- Outbox Pattern.
- Idempotência no consumo de mensagens.
- Retry e dead letter queue.
- Tracing distribuído.
- Autenticação e autorização.
- CI/CD.
- ADRs curtos.
- Estratégia clara para refatoração de legado.
- Paginação e filtros.
- Boa organização do frontend.
- Boa explicação sobre uso de IA.

08 Como sua entrega será analisada

Dimensões de avaliação técnica.

A avaliação buscará consistência entre funcionalidade entregue, organização técnica, documentação e capacidade de explicar as escolhas realizadas.

Dimensão	O que será observado
Funcionalidade entregue	Fluxos principais funcionando e aderentes às regras de negócio.
Organização do backend	Separação de responsabilidades, clareza de camadas e modelagem.
Regras de negócio	Consistência da matrícula, controle de vagas e cancelamento.
Testes	Cobertura das regras críticas e qualidade dos cenários.
Frontend	Organização, fluxo de uso e tratamento de erros.
Documentação	README, documentação da API e documentação arquitetural.
Arquitetura/desacoplamento	Modularização, separação de contextos e decisões justificadas.
Mensageria/eventos	Eventos de domínio, consumidores, tratamento de falhas e consistência.
Observabilidade/operação	Logs, rastreabilidade, health checks e métricas básicas.

► Sinais qualitativos valorizados

- Clareza de raciocínio e simplicidade nas escolhas.
- Capacidade de explicar o próprio código.
- Cuidado com erros, validações e transações.
- Consistência entre README, arquitetura e implementação.
- Pragmatismo: nem código improvisado, nem complexidade gratuita.

09 Entrevista técnica após a triagem

A conversa será baseada no código entregue.

Caso sua entrega avance para a etapa técnica, a entrevista buscará confirmar autoria, profundidade técnica e maturidade nas decisões. Exemplos de pontos que poderão ser explorados:

- Mostre o fluxo de confirmação de matrícula.
- Onde está protegida a regra de limite de vagas?
- O que acontece se duas pessoas tentarem se matricular ao mesmo tempo na última vaga?
- Como você testou essa regra?
- Se esse sistema atendesse muitas instituições ao mesmo tempo, o que mudaria?
- Como você separaria esse módulo em outro serviço?
- Como monitoraria erro de consumo de mensagem?
- Qual decisão você tomaria diferente com mais tempo?
- Que parte foi feita com ajuda de IA?
- Explique um trecho crítico sem consultar documentação.

Boa sorte!

Valorize clareza, consistência, simplicidade e justificativa técnica.